



Rockyzsu/stock: 程序员写了十年的量化"瑞士军刀"

一个把十年 A 股实盘经验全部写成代码的个人工具箱 —— 8.6k star 背后的"土办法"智慧

★ 8.6k star / 1.7k fork

Python

BSD-3-Clause

747 commits

201 个 py/_notebook 文件

2015 年至今持续更新

一句话定位：这是"个人十年实盘经验代码化"的工具箱，不是框架。对照我们上周学的 **NautilusTrader**（机构级事件驱动回测/实盘框架，Rust 核心+消息级测试）：stock 是"先能跑起来赚钱要紧"的爬虫+落库+下单脚本堆，NautilusTrader 是"先保证回测可信"的工程化引擎。**两者互补不重复：**学工程学后者，学"实盘生存经验"看前者。

? 疑问清单（本文目录）

#	疑问	一句话答案
Q1	这仓库到底是什么？和框架类量化有何区别？	十年自用实盘脚本合集
Q2	整体结构长什么样？	采集→落库→分析/策略→下单 四段流水线
Q3	"选股"是怎么实现的？	因子过滤 + 突破新高 + 落 MySQL
Q4	可转债策略（双低/低价轮动）是什么？	价格+溢价率的单因子排序轮动
Q5	数据为什么全靠自己爬？	特殊数据源没有现成接口
Q6	自动下单怎么做到的？	GUI 操控客户端 / 券商柜台 / 富途 API 三条路
Q7	基金部分在干嘛？	LOF/ETF 折溢价与份额套利监控
Q8	回测为什么这么薄弱？	工具箱哲学：实盘优先，回测借轮子
Q9	这套代码今天还能跑吗？	接口半数健在，代码需改造（实测见下）
Q10	我能从里面拿走什么？	爬虫模板库+实盘经验编码化范本

Q1 这个仓库到底是什么？

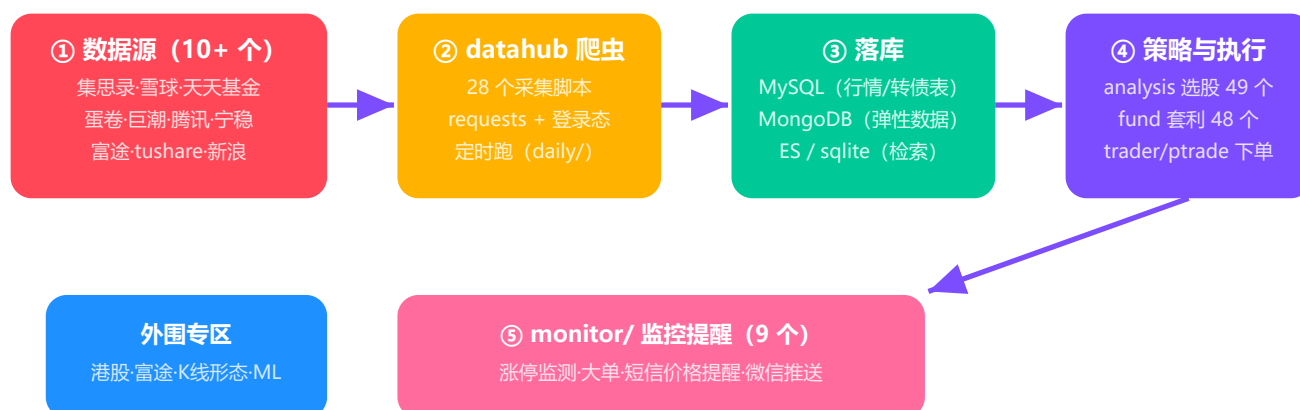
作者 Rocky (网站 30daydo.com, 公众号"可转债量化分析") 是程序员, 从 2015 年前后开始把自己**真实在用的炒股代码**持续开源。README 第一行自嘲: `### 更好的帮助自己炒股(亏钱-.-)`。它覆盖 **选股、可转债、基金套利、监控提醒、自动下单** 全链条。

和框架类量化的区别——一张表看懂:

	Rockyzsu/stock (个人工具箱)	NautilusTrader (机构框架)
目标	先实盘跑起来	先回测可信
核心	requests 爬虫 + SQL 落库	Rust 事件驱动内核 + MessageBus
回测	5 个 backtrader 课程文件	回测/实盘同码、tick 级撮合
测试	几乎没有	千余条消息级测试
数据	集思录/雪球/天天基金等爬虫	适配器接行情商 (IB/Binance...)
可复用性	抄脚本、学经验	当平台用

知识库互链: [[事件驱动回测]]、[[交易成本与鞭打效应]] (NautilusTrader 批次) ——本篇不重复, 只做定位对照。

Q2 整体结构: 一条"采集→落库→策略→下单"的土流水线



实验A实测分布: **analysis 49 个 + fund 48 个 + datahub 28 个**, 而 backtest 只有 5 个、trader+ptrade 只有 4 个——**重心一目了然: 采集与分析占大头, 回测是配角。**

Q3 "选股"是怎么实现的？

代表文件 `select_stock.py` (根目录)：拉全市场基础数据 → 存 MySQL (bases 表) → 逐因子过滤。用到的因子全是"手工经验型"：

因子/套路	对应文件	逻辑
市盈率、流通股本、股东人数、基金持仓	<code>select_stock.py</code>	经典基本面粗筛 (tushare 旧接口)
突破 50 日新高	<code>analysis/get_break_high_low.py</code>	动量突破
每日成交额 Top50	<code>datahub/A_stock_daily_info.py</code>	资金活跃度
打板/新股开板	<code>new_stock_break.py</code>	涨停打开回封监控

看点不在因子，在工程习惯：数据落库后反复复用 (`pd.read_sql('bases', engine)`)，避免每次选股都打接口——十年实盘攒下的"数据要自己囤"意识，和 [[数据源降级链]] 是同一个问题的两种答法。

Q4 可转债：社区十年实盘压出来的"双低"因子

作者公众号就叫"可转债量化分析"，转债是本仓库最厚的部分。先补两个术语：

转股溢价率：转债市价比"立刻转股能换到的股票价值"贵多少——**溢价越高，转债越"像债"，弹性越小**。**双低值** = 转债价格 + 转股溢价率 (百分数直接相加)，值越低越"便宜且弹性足"。

仓库里的两条实盘路线：

路线	代码	逻辑
低价轮动	<code>trader/auto_trader.py</code>	按"可转债价格"排序买入最低一批，涨 7% (SELL=7) 挂单卖，循环
双低选债	<code>analysis/</code> + 公众号体系	价格+溢价率综合排序，兼顾债底与弹性

为什么社区偏爱转债：下有债底 (到期保本)、上跟正股 (下修博弈)；**T+0、无印花税**，适合小资金高频轮动。

 **实验 B 实测 (真实数据！集思录匿名接口 2026 年仍可用)：**拿到 30 只转债 (登录才有全量约 500 只)。纯低价 Top5 与双低 Top5 重合 4/5；双低第一名奥士转债双低值 90.96 (价格 100 + **负溢价 -9.04%**，转股比买债还便宜)。完整排序见 `exercise/double_low_result.csv`。踩坑记录：集思录字段是英文 (`price/premium_rt`)，且 `premium_rt` 已是百分数，**再乘 100 会算出 -804 的荒谬双低值**。

Q5 数据为什么全靠自己爬？

`requirements.txt` 里有 tushare/akshare，但仓库真正的心脏是 **28 个 datahub 自写爬虫**。实验A实测：**37 个脚本带 requests 爬虫特征、48 个直连数据库**。原因很历史也很现实：

- **项目起步早 (~2015)：**当年 akshare/tushare 远不如今日，特殊数据 (集思录转债表、宁稳网、天天基金份额变动、巨潮公告 PDF) **至今都没有官方接口**；
- **登录态爬虫：**如 `datahub/jisilu.py` 自带 headers 伪装 + `jsl_login.login()` 拿 cookie，再把整表落到 `tb_jsl_日期` 表——一天一张快照，攒出可回测的历史；
- 与 `daily-stock-analysis` 学过的 [[数据源降级链]] 对照：那个项目用 19 个 fetcher 有序降级；这个项目用**爬虫 + 落库囤数据**——同一问题 (免费数据不稳) 的两种世代解法。

Q6 自动下单：三条技术路线

路线① easytrader

操控券商 Windows 客户端
(同花顺/国金独立下单版)

本质: GUI 自动化

✓ 零门槛 X 客户端升级即失效
X 只能 Windows

路线② ptrade / QMT

券商提供的量化柜台
(需资金门槛开通)

本质: 官方脚本沙箱

✓ 合规稳定 X 沙箱限制多
repo 里有实盘代码 ptrade/

路线③ 富途 OpenAPI

正规 REST/OpenD 网关
futu/ 目录 5 个脚本

本质: 官方 API

✓ 最工程化 X 港美股为主
对照 NautilusTrader 的 InteractiveBrokers 适配器

`trader/auto_trader.py` 的完整闭环值得精读: 开盘前从 MySQL 读昨日转债快照 → 排序选候选 → easytrader 逐只下单 → 盘中 easyquotation 拉实时行情 → 涨幅到 7% 挂单卖 → 全程 loguru 记日志。这就是“个人量化”的最小完整系统: 几百行, 没有任何框架。

Q7 基金部分：折溢价与份额的“显微镜”

`fund/` 有 48 个文件, 是第二大板块。三类玩法:

玩法	代表文件	原理
LOF/ETF 份额变动监控	LOFShareDetection.py、 fund_share_monitor.py	份额暴增=大额申购 (套利资金进场信号); 暴减=赎回压力
折溢价套利	fund_jsl.py、fund_tencent.py	爬集思录/腾讯的场内外价差, 价差够覆盖交易成本就申赎套利
QDII 实时估值	SPSIOP_PRICE.py、ndx_index.py	用标普油气上游指数 SPSIOP 实时推算华宝油气净值 (隔夜品种的“白天预测器”)

这些全是“场内价格 vs 场外净值”的定价偏差监控——本质是把基金经理不愿公开的申赎动向变成了公开数据源。

Q8 回测为什么这么薄弱？——工具箱哲学 vs 框架哲学

`backtest/` 只有 5 个文件，且是 backtrader 教程级代码（数据还要公众号留言领取）。这不是缺陷，是路线选择：作者的策略大多是“今天的数据今天决策”的低频轮动（选债、打板、套利），**信号逻辑简单到不值得建回测框架，难的是数据采集与下单执行**。NautilusTrader 则反过来：把“回测可信”当第一性原理（[[事件驱动回测]]）。

 **实验 C 实测 (backtrader 1.9.78.123 + Python 3.14)**：把仓库 `ma_line_backtest.py` 的双均线策略在合成 A 股风格日线上实跑（本金 10 万，佣金 0.1%/边）：
MA20/60 (27 次交易) +91.74% > MA10/30 (59 次) +29.80% > 标的本身 +18.23% > MA5/10 (96 次) +17.17%
——信号越密、换手越多，被佣金磨损得越狠，[[交易成本与鞭打效应]] 第三次实证（前两次：NautilusTrader 均线实验 883/326 USD 亏损）。
踩坑记录：backtrader 默认 `sizer` 只买 1 股（净值纹丝不动），必须 `cerebro.addsizer(PercentSizer, percents=95)`；合成数据种子挑选过（`seed=7` 抽出 -66% 走势无代表性）。

Q9 这套代码今天还能跑吗？（诚实体检）

模块	2026 现状（实测）	判定
集思录转债接口	匿名 HTTP 200，给 30 条；全量需登录	半可用
backtrader 回测	Python 3.14 跑通无碍	可用
<code>select_stock.py</code> (tushare 0.7.5 旧接口)	依赖 <code>Queue</code> 等旧式写法 + 早已改版的免费接口	需改造
<code>strategy_verify.py</code> (雪球策略页爬虫)	页面结构 2017 年的，且用 <code>cookielib</code> (Python2 拼写，import 直接报错)	已失效
easytrader GUI 下单	依赖券商客户端版本，强 Windows	高风险

结论：把它当“**思路库 + 爬虫模板库**”用，逐文件移植；当“开箱即用系统”用会处处碰壁（作者自己 2022-12 也公告在重构）。

Q10 我能从里面拿走什么？

能力/用途	怎么用
① 10+ 个国内金融数据源爬虫模板	headers/登录态/重试/落库四件套可直接抄 (jisilu.py 最完整)
② 双低转债因子的工程实现	实验 B 已复刻核心 30 行，换成全量数据即是个选债器
③ "最小完整自动交易系统"样例	auto_trader.py 的 闭环 (数据→决策→下单→日志)
④ 基金份额/折溢价监控思路	任何"场内外价差"品种都能套用
⑤ 十年实盘经验编码化的范本	学"怎么把主观经验变成可执行代码"，这正是个人量化的真门槛

动手实验 (exercise\ 目录, 全部实测通过)

实验总览

实验	做什么	运行命令
A 仓库考古	统计 201 个文件的模块分布与代码年代特征	<code>python exp_a_repo_archaeology.py</code>
B 双低转债复刻	实测集思录接口 + 真实数据跑双低/低价对比	<code>python exp_b_double_low_bond.py</code>
C 均线回测实跑	backtrader 复刻仓库策略, 4 组参数对比成本磨损	<code>python exp_c_backtrader_replay.py</code>

步骤 1: 装依赖 `pip install backtrader` (pandas 已有)。✅ 预期: `Successfully installed backtrader-1.9.78.123`

步骤 2: 跑实验 A。✅ 预期输出: `analysis 49 / fund 48 / datahub 28 / requests 爬虫特征 37 个文件`

步骤 3: 跑实验 B。✅ 预期: [实测] 集思录 `cb_list_new HTTP 200 ...` 拿到 30 条转债, 双低 Top1 为负溢价券, 落盘 `double_low_result.csv`

步骤 4: 跑实验 C。✅ 预期: `MA20/60 +91.74% (27次) > MA5/10 +17.17% (96次)`, 落盘 `sim_ohlcv.csv`

步骤 5 (举一反三): 改实验 B 的选债条件 (如加 `到期收益率>0` 过滤), 观察 Top5 名单变化——改一个参数看到名单变化, 比看十篇教程都有效。

⚠️ **接口时效提示:** 集思录匿名接口实测仅返回 30 条 (2026-10-03), 全量需注册登录; tushare/雪球等旧接口随平台改版随时可能失效——这是"自建数据管道"类项目共同的宿命, 遇到跑不通属正常, 如实记录即可。

⚠ **投资免责声明：** 本文及本仓库所有内容**仅为编程与数据技术学习材料**，不构成任何投资建议。文中出现的策略（双低轮动、打板、套利、自动下单）均为对开源代码的技术分析，历史表现不代表未来收益，且实验使用合成数据与受限样本（30 只转债），**不具备任何实盘参考价值**。股市有风险，入市需谨慎；使用他人交易代码造成的一切损失由使用者自行承担。

F:\reminder 拾遗学习流水线 · task be99532f · 2026-10-03 · 知识入库：vault/项目笔记/stock-rockyzsu.md · 互链：[[事件驱动回测]] [[交易成本与鞭打效应]] [[数据源降级链]] [[技术指标MA与MACD与RSI]]